DISEASE-AWARE PADDY HARVEST YIELD ESTIMATION AND TREATMENT DECISION SUPPORT SYSTEM FOR SRI LANKAN PADDY CULTIVATION

Weerasinghe W.A.H.V

(IT22266446)

BSc (Hons) in Information Technology
Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DISEASE-AWARE PADDY HARVEST YIELD ESTIMATION AND FINANCIAL DECISION SUPPORT

Weerasinghe W.A.H.V

(IT22266446)

Dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature: …… Date: ……

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the supervisor: ……. Date: ……

(Mrs. Chathurangika Kahandawarachchi)

ABSTRACT

Paddy cultivation in Sri Lanka is affected by environmental variability, crop growth conditions and disease, creating uncertainty in expected harvest outcomes. The individual research component of Project R26-IT-066 addresses this problem through a disease-aware harvest yield estimation and treatment decision-support artifact. The implemented component uses field and crop-condition variables for baseline yield prediction and disease type, severity and growth stage for disease-related loss estimation. The inspected software artifact contains a Random Forest regression model for base-yield estimation, an XGBoost regression model for disease-loss estimation, a Python Flask prediction service, a Flutter yield-treatment interface and Firebase telemetry integration. The decision-support service calculates base yield, predicted loss, untreated yield and treated yield, and returns a treatment decision through a REST endpoint. The current treated scenario assumes that treatment reduces predicted disease loss by 60%; this is explicitly treated as a software assumption rather than a validated agronomic treatment effect. The research follows a Design Science Research approach and evaluates the artifact at model, API, integration and user levels. Implementation inspection confirms the major analytical and software components, including model artifacts, feature construction, scenario equations and the client interface. However, the supplied evidence set does not contain a verified final experiment log with the dataset size, held-out R^2/MAE/RMSE values, measured API latency or completed farmer usability scores. These values must therefore be inserted from the completed experimental records before final submission rather than being inferred from proposal targets. The resulting artifact provides a modular foundation for linking disease information to harvest-oriented decision support within a wider smart-agriculture system.

Keywords - ***paddy yield estimation, disease-aware prediction, Random Forest, XGBoost, IoT, disease severity, treatment decision support, Sri Lankan paddy***

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. Chathurangika Kahandawarachchi, for her guidance, feedback and encouragement throughout the research and preparation of this dissertation.

I acknowledge the contributions of the other R26-IT-066 group members whose IoT, crop recommendation/market and disease-analysis components provide supporting information for the individual harvest-yield component. Their collaboration was important for understanding the integrated Decision Support System.

I also acknowledge the Department of Census and Statistics Sri Lanka, the International Rice Research Institute and the other academic and institutional sources used in this research. I am grateful for the technical resources and learning environment provided by the Sri Lanka Institute of Information Technology.

Finally, I thank my family, friends and colleagues for their continuous encouragement and support throughout the completion of this research.

Table of Contents

DECLARATION III

ABSTRACT IV

ACKNOWLEDGEMENT V

LIST OF ABBREVIATIONS 9

1. INTRODUCTION 1

1.1 Background and Context 1

1.2 Literature Review 2

1.3 Comparison of Similar Work 5

1.4 Research Gap 6

1.5 Research Problem 7

1.6 Research Objectives 9

2. METHODOLOGY 13

3. RESULTS AND DISCUSSION 29

3.1 Implemented Artifact Results 29

3.2 Base-Yield Model Artifact Analysis 31

3.3 Disease-Loss Model Artifact Analysis 36

3.4 Scenario Calculation Verification 37

3.5 API Verification 37

3.6 Functional Test Design 38

3.7 Non-Functional Evaluation 40

3.8 Research Findings 41

3.9 Comparison with Existing Approaches 47
3.10 Discussion of Research Objectives 47
3.11 Limitations 49
3.12 Practical Implications 49
3.13 Commercialization Findings 49
3.14 Chapter Summary 50
4. CONCLUSION 51
4.1 Summary of the Research 51
4.2 Achievement of the Aim 51
4.3 Individual Contribution 51
4.4 Key Technical Contribution 52
4.5 Limitations and Required Corrections 52
4.6 Future Work 52
4.7 Final Conclusion 53
REFERENCES 54
GLOSSARY 56
APPENDICES 58

LIST OF FIGURES

Figure 2.1:System overview diagram 15
Figure 2.2:System overview diagram 21
Figure 2.3:Implemented IT22266446 Component Architecture.....23
Figure 3.1:Distribution of Paddy Yield Values in the Dataset.....32
Figure 3.2: Boxplot of Paddy Yield Values.....33
Figure 3.3: Correlation Heatmap of Yield and Input Features 34
Figure 3.4: Actual vs. Predicted Paddy Yield Using the Random Forest Model.....35
Figure 3.5: Feature Importance of the Random Forest Yield Prediction Model 36

LIST OF TABLES

Table 1.1: Comparison of related research and agricultural systems.....6
Table 1.2: Research Gap Analysis 7
Table 1.3: Relationship between the research objectives.....12
Table 2.1: Mapping of Research Phases to Objectives 14
Table 2.2: Input Features and Their roles 16
Table 2.3: Functional Requirements 24
Table 2.4: Non-Functional Requirements.....25
Table 3.1: Functional and integration test cases 40
Table 3.2: Evidence - based evaluation summary.....41
Table 3.3: Baseline-Yield Model Comparison.....42
Table 3.4: Objective achivment matrix.....49

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
DCS	Department of Census and Statistics
DOA	Department of Agriculture
DSR	Design Science Research
DSS	Decision Support System
IRRI	International Rice Research Institute
IoT	Internet of Things
MAE	Mean Absolute Error
ML	Machine Learning
NPK	Nitrogen, Phosphorus and Potassium
RF	Random Forest
RMSE	Root Mean Squared Error
REST	Representational State Transfer
SES	Standard Evaluation System
UI	User Interface
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

1.1 Background and Context

Agriculture is increasingly being transformed by sensing, connectivity, machine learning and decision-support technologies. Smart agriculture is commonly described as the use of information and communication technologies to observe farm conditions, analyse data and support timely management decisions. The literature shows that smart farming is not only a matter of adding sensors to a farm; it also requires an information architecture through which heterogeneous data can be converted into actionable decisions [15], [16]. This distinction is important for paddy cultivation because farmers are often confronted with multiple interacting variables rather than a single isolated problem.

Paddy is strategically important to Sri Lanka for food security, rural livelihoods and agricultural activity. National agricultural statistics are reported across the Maha and Yala cultivation seasons, and production varies with cultivated extent, weather, water availability, agronomic practices and crop health. The Department of Census and Statistics provides district- and season-level paddy statistics that can support data-driven analysis [1]. These historical statistics are useful for modelling because they provide a target variable harvest yield against which environmental and crop condition variables can be related.

Yield estimation is nevertheless difficult because paddy yield is an outcome of interacting biological and environmental processes. Soil moisture, temperature, humidity, nutrient availability, growth stage and variety can influence crop development. The machine-learning literature indicates that non-linear models are well suited to agricultural prediction problems where multiple variables interact [17]. Tree-based ensemble models are particularly useful because they can represent non-linear relationships without requiring a fully specified analytical crop-growth equation. Random Forest provides an ensemble of decision trees and is robust to complex relationships and noise [12]. XGBoost extends gradient-tree boosting with scalable and regularized learning mechanisms [13].

Disease introduces an additional layer of uncertainty. A disease detection system may identify Rice Blast or Brown Spot and, in a more advanced system, estimate severity. However, a disease label by itself does not directly answer the farmer's harvest question. The farmer needs to know how the detected condition may translate into expected yield reduction and whether an intervention is worthwhile. The submitted individual proposal therefore positioned disease-aware yield estimation as the analytical layer connecting disease information to harvest outcomes. This thesis develops that layer as an individual component of the wider Smart Agriculture Decision Support System.

The group architecture contains four complementary research components. IT22178718 focuses on IoT field monitoring and fertilization guidance; IT21811104 focuses on crop recommendation and market analysis; IT22261632 focuses on paddy disease detection and severity analysis; and IT22266446 focuses on disease-aware harvest yield estimation. The individual component does not replace the other modules. Instead, it consumes field and disease information and transforms them into harvest-oriented outputs. This separation is important when evaluating the individual contribution because the thesis must distinguish the student's artifact from the functionality developed by the other group members.

1.2 Literature Review

Machine learning has become a significant research direction in agriculture because agricultural processes contain complex, non-linear and context-dependent relationships. Liakos et al. reviewed machine-learning applications in agriculture and identified uses including yield prediction, disease detection and crop management [17]. Their review demonstrates that machine learning can support both predictive and prescriptive agricultural tasks, but model usefulness depends strongly on the quality and representativeness of the data. For the present research, this supports the use of supervised regression for yield estimation while also highlighting the need to report data sources and validation procedures explicitly.

Random Forest is an ensemble learning method based on multiple decision trees trained with randomized sampling and feature selection [12]. Its ability to model non-linear relationships and its relative robustness make it suitable for agricultural datasets containing mixed continuous and categorical variables. In the implemented component, a Random Forest regression artifact is used for the base-yield prediction. The deployed model bundle stores both the trained model and the expected feature-column order, allowing the inference service to reconstruct the model input consistently.

XGBoost is a gradient-boosting framework that constructs an ensemble of decision trees sequentially and optimizes a regularized objective [13]. The original XGBoost study emphasized scalability, sparse-data handling and efficient tree learning. In the implemented artifact, XGBoost is used for the disease-related loss prediction stage. This is conceptually appropriate because disease loss can depend non-linearly on severity, disease type and growth stage. The model receives a compact set of disease-related features rather than the full field vector used by the base-yield model.

IoT-enabled smart agriculture provides an important source of near-real-time field information. Friha et al. surveyed IoT technologies for smart agriculture and highlighted the role of sensing, communication and data processing in intelligent farming [15]. The group project follows this general architecture by obtaining field measurements through an IoT module and making those measurements available to downstream decision components. For the individual yield component, the relevant variables include soil moisture, temperature, humidity and nutrient measurements. The Flutter client also contains a Firebase service capable of reading recent telemetry and populating nutrient values in the yield-treatment interface.

Disease assessment and disease severity are important because yield impact is not constant across all disease conditions. The proposal uses the International Rice Research Institute's Standard Evaluation System for Rice as a reference for disease severity and yield-loss relationships [2]. Research on rice disease has also examined severity assessment and the relationship between disease intensity and crop impact [3], [7], [8], [10]. The individual

component adopts the principle that severity and crop stage should be treated as explanatory variables for disease-related loss rather than treating disease as a binary variable.

Sri Lankan agricultural decision-support platforms demonstrate the value of localized digital services. The proposal identifies eKeth, GeoGoviya and the Department of Agriculture's Krushi Advisor as related systems [4]–[6]. These systems show that mobile or digital agricultural advisory services are feasible in the Sri Lankan context. However, the individual research gap is narrower: the proposed contribution is the integration of baseline yield prediction, disease-related loss estimation and scenario comparison within the wider smart-agriculture DSS. This distinction prevents the thesis from claiming that no agricultural software exists in Sri Lanka; instead, it identifies a specific integration problem.

Yield-prediction studies outside Sri Lanka also demonstrate the use of machine-learning regression for rice and other crops [9], [11]. These studies support the feasibility of using historical yield and environmental variables for prediction. At the same time, a model trained in one geographical context may not transfer directly to another because varieties, climate, management practices and measurement processes differ. The proposal therefore emphasized Sri Lankan district and seasonal data and identified BG300 and AT362 as target varieties. The thesis retains this local-context requirement while describing only the implementation characteristics that are verified in the source code.

The broader smart-farming literature also emphasizes that data infrastructures and decision systems create socio-technical issues involving governance, privacy, interoperability and business models [16]. This is relevant to the proposed system because the value of the yield component is not limited to prediction accuracy. The component must also provide an interpretable interface, communicate through APIs, manage input validation, and operate within a wider data pipeline. These requirements are considered in the methodology, implementation and evaluation sections.

1.3 Comparison of Similar Work

Table 1 compares representative approaches relevant to the individual component. The comparison is intentionally organized around what each approach contributes and what remains outside its scope. The purpose is to establish a defensible research gap rather than to make an unsupported claim of absolute novelty.

Related work / system	Main contribution	Relevance to this research	Scope not established by cited source
IRRI Standard Evaluation System for Rice [2]	Provides standardized rice-disease and crop assessment guidance.	Offers a domain reference for describing disease severity and interpreting disease-related crop impact.	It is a reference framework, not by itself an integrated, real-time yield and treatment-decision application.
Rice disease severity research [3], [7], [8], [10]	Studies disease identification, severity assessment, or disease impact on rice.	Supports treating disease type, severity, and crop stage as meaningful factors in disease-aware analysis.	The cited studies do not, as a group, establish the exact integrated workflow implemented in this project.
Rice/crop yield prediction studies [9], [11]	Investigate data-driven estimation of crop yield using crop, environmental, or remote-sensing variables.	Supports the feasibility of regression-based baseline-yield estimation.	The literature review does not establish that these approaches combine the project's disease-loss and treated/untreated scenario outputs.
Sri Lankan agricultural platforms: eKeth [4]	Provides a digital platform intended to support paddy cultivation activities.	Demonstrates the relevance of mobile/digital agricultural support in the local context.	The reviewed proposal does not document a comparable disease-severity-to-yield-loss scenario engine.
GeoGoviya [5] and Krushi Advisor [6]	Provide agricultural information, digital services, or advisory support in Sri Lanka.	Show the practical role of localized digital agricultural information and advice.	The sources reviewed here do not verify the same combination of baseline yield, disease-loss estimation, and treatment scenario comparison.

IoT and smart-farming systems [15]-[17]	Describe sensing, connected agricultural data, and data-driven smart-farming architectures.	Support the architecture of using field data as inputs to downstream decision support.	General smart-farming architecture alone does not validate this project's disease-loss model or treatment-effect assumption.
Proposed IT22266446 component	Combines a baseline-yield model, disease-loss model, scenario calculations, API, and Flutter interface.	Targets an integrated workflow from field/crop and disease inputs to untreated and treated yield estimates.	The disease-loss evaluation and current prediction-button issue remain unresolved; the treatment improvement assumption is not experimentally validated.

Table 1.1: Comparison of related research and agricultural systems

1.4 Research Gap

Existing studies have applied machine learning techniques to predict crop yields using environmental, soil, and agricultural data [1], [2]. However, predicting the expected yield alone may not fully support farmers when paddy cultivation is affected by diseases. Disease severity can reduce the expected harvest, making it important to consider both the baseline yield and the potential yield loss.

This research addresses this gap by developing a disease-aware paddy harvest yield estimation and decision-support component for Sri Lankan paddy cultivation. The component combines environmental conditions, soil nutrient values, paddy growth stage, season, variety, and disease information to estimate baseline yield and disease-related yield loss. It then compares untreated and treatment scenarios to provide a decision-support output.

Area	**Existing limitation**	**Contribution of this research**
Yield prediction	Expected yield alone may not reflect disease-related losses.	Estimates baseline yield and disease-related yield loss separately.

Disease impact	Disease information may not be directly connected to yield estimation.	Includes disease type and severity as model inputs.
Scenario comparison	Farmers may lack a simple comparison of possible harvest outcomes.	Compares estimated untreated and treated yields.
Decision support	Prediction results may not directly provide an understandable recommendation.	Generates a treatment-related decision-support output.
System integration	Model outputs may not be easily accessible through an application.	Integrates the prediction service with the Flutter application and Firebase-based data services.

Table 1.2: Research Gap Analysis

Therefore, the main research gap addressed is the integration of disease-aware yield estimation and treatment-scenario comparison into an application-based decision-support component designed for Sri Lankan paddy cultivation. The treatment scenario is based on the current implementation's assumption that treatment reduces disease-related yield loss by 60%; this assumption should not be interpreted as experimentally validated treatment effectiveness.

1.5 Research Problem

Paddy cultivation in Sri Lanka is affected by variations in environmental conditions, soil nutrients, crop growth stages and plant diseases. Diseases such as Rice Blast and Brown Spot can affect different parts of the rice plant and reduce the quality and quantity of the final harvest. The severity and impact of these diseases can vary depending on crop conditions and the stage of cultivation [3]. Therefore, estimating the expected harvest

requires consideration of both the general growing conditions and the potential impact of disease.

Although machine-learning models can be used to estimate crop yield and digital systems can provide disease-related information, these outputs do not automatically provide farmers with a clear understanding of how a detected disease may affect their expected harvest. A farmer may receive information about a disease but still lack an estimated comparison of the expected yield under different disease-management scenarios. This creates a decision-support problem because the farmer needs to understand the potential consequences of leaving the disease untreated and the possible outcome of applying treatment.

The main research problem addressed by this study is the difficulty of translating field conditions and disease-related information into understandable, comparable paddy harvest-yield estimates. In particular, the research investigates how environmental variables, soil nutrient values, crop growth stage, paddy variety, cultivation season, disease type and disease severity can be used to estimate baseline yield and disease-related yield loss. It also considers how these estimates can be used to compare untreated and treated yield scenarios through an application-based decision-support component.

To address this problem, the individual component of Project R26-IT-066 integrates a Random Forest model for baseline yield estimation and an XGBoost model for disease-related loss estimation. The resulting predictions are processed to calculate untreated and treated yield scenarios and generate a treatment-related decision-support output. The component is connected to the wider system through a Flask-based prediction service and a Flutter user interface, with Firebase integration supporting the retrieval of available field data.

However, the reliability of the resulting recommendation depends on the accuracy of the model predictions and the validity of the assumptions used in scenario calculations. In the current implementation, the treated scenario assumes a 60% reduction in predicted

disease-related yield loss. This assumption has not been established as a field-validated treatment effect and must therefore be considered when interpreting the output. Accordingly, the research problem includes not only developing an integrated prediction mechanism but also identifying its limitations and evaluating its behaviour using actual experimental evidence.

Therefore, this research focuses on developing and evaluating a disease-aware paddy harvest-yield estimation and decision-support component that converts available field and disease information into baseline yield estimates, disease-related loss estimates and comparable treatment scenarios for Sri Lankan paddy cultivation.

1.6 Research Objectives

1.6.1 Main Objective

The main objective of this research is to develop and evaluate a disease-aware paddy harvest yield estimation and decision-support component that uses environmental conditions, soil nutrient information, crop characteristics and disease-related inputs to estimate expected harvest yield and compare untreated and treated scenarios for Sri Lankan paddy cultivation.

The component aims to combine machine-learning-based yield prediction with disease-related yield-loss estimation to provide farmers with a clearer understanding of potential harvest outcomes under different disease-management scenarios. It is developed as an individual component of the R26-IT-066 Smart Agriculture Decision Support System.

1.6.2 Specific Objectives

The following specific objectives define the main development and evaluation activities of the research.

SO1: To develop a machine-learning model for baseline paddy harvest yield estimation.

This objective focuses on estimating the expected paddy yield using relevant environmental, soil and crop-condition variables. The implemented component uses a Random Forest model for baseline yield prediction. The input features include soil moisture, temperature, humidity, nitrogen, phosphorus, potassium, crop growth stage, cultivation season and paddy variety. The model output provides the baseline yield estimate used in subsequent scenario calculations.

SO2: To estimate disease-related paddy yield loss using disease and crop information.

This objective focuses on estimating the potential percentage of yield loss associated with the disease conditions affecting the crop. The implemented component uses an XGBoost model and includes disease type and disease severity alongside relevant crop-condition information. The predicted loss percentage is used to estimate the yield that may remain if the disease is left untreated.

SO3: To develop a scenario-based mechanism for comparing untreated and treated harvest yields.

This objective aims to combine the baseline yield and predicted disease-related loss into comparable yield scenarios. The implemented calculation estimates untreated yield by applying the predicted loss percentage to the baseline yield. It also calculates treated yield using an assumed reduction in disease-related loss. These estimates are used to generate a treatment-related decision-support output. The treatment-effect assumption must be evaluated separately and should not be interpreted as proven treatment effectiveness.

SO4: To integrate the prediction and scenario calculations into an application-based decision-support component.

This objective focuses on making the prediction outputs accessible through the wider R26-IT-066 system. The implementation includes a Flask-based prediction service, the POST /decision-support/predict endpoint and a Flutter interface for yield-treatment information. Firebase integration supports access to available field telemetry. This objective also requires testing the input handling, prediction workflow, output generation and integration behavior.

SO5: To evaluate the developed component against relevant technical and functional requirements.

This objective focuses on assessing the prediction models and the operation of the integrated component using documented experimental and testing evidence. Model evaluation should use appropriate regression metrics, such as (R^2), MAE and RMSE, where supported by the available test data. Functional testing should verify whether the service accepts valid inputs, produces the expected output fields and handles invalid or incomplete inputs appropriately. Any performance, usability or accuracy claims must be supported by actual evaluation results.

1.6.3 Specific Objectives and Measurable Success Criteria

Table 1.3 presents the relationship between the research objectives, the planned evaluation criteria and the evidence required to assess their achievement.

Objective	Measurable success criteria	Required evidence
SO1 – Baseline yield estimation	Generate baseline yield predictions and evaluate model performance against actual test-set values.	Test dataset, predictions and regression metrics.
SO2 – Disease-related yield-loss estimation	Generate loss estimates and evaluate them against reference or labelled loss values.	Validation data, predicted loss values and regression metrics.

SO3 – Scenario comparison	Calculate baseline, untreated and treated yield outputs consistently and verify the scenario equations.	Test cases, calculation checks and scenario outputs.
SO4 – System integration	Verify that the prediction API and Flutter interface exchange and display the required information correctly.	API test logs, application screenshots and integration tests.
SO5 – Component evaluation	Complete documented model and functional testing and compare the findings with the defined requirements.	Evaluation results, test records and objective-achievement analysis.

Table 1.3: Relationship between the research objectives

Note: The success criteria in Table 1.3 describe the evidence needed to evaluate the objectives; they do not indicate that every objective has already passed evaluation. Numerical performance targets stated in the original proposal should be reported separately and confirmed using actual experimental results.

1.6.4 Summary

The research objectives address the development of a baseline yield prediction model, estimation of disease-related yield loss, comparison of untreated and treated yield scenarios, integration with an application interface and evaluation of the resulting component. Together, these objectives establish a structured approach to developing and assessing the individual contribution of IT22266446 within the Smart Agriculture Decision Support System.

2. METHODOLOGY

This chapter describes the methodology used to design, implement and evaluate the Disease-Aware Paddy Harvest Yield Estimation and Decision Support component of Project R26-IT-066. It presents the research approach, system architecture, data inputs, feature preparation, machine-learning models, yield-scenario calculations, software implementation, testing strategy and commercialization aspects. The methodology distinguishes the original research proposal from the functionality verified in the current implementation.

2.1 Research Approach

This research adopts a Design Science Research (DSR) approach because its main output is a software artefact developed to address a practical agricultural decision-support problem. DSR provides a structured approach to identifying a problem, defining objectives, designing and developing a solution, demonstrating its operation and evaluating its performance [1]. The approach is appropriate for this research because the component combines machine-learning models, scenario calculations and an application interface into a functional software solution.

The research process consists of five main stages:

1. **Problem identification:** Identify the difficulty of translating paddy field conditions and disease information into useful harvest-yield estimates.
2. **Objective definition:** Establish objectives for baseline yield estimation, disease-related yield-loss prediction, scenario comparison, application integration and evaluation.
3. **Design and development:** Prepare the prediction workflow, integrate the trained model artefacts, implement yield calculations and connect the prediction service to the application interface.

4. **Demonstration:** Execute the prediction workflow using the implemented backend and Flutter interface.

5. **Evaluation:** Assess model performance, calculation consistency, API behaviour and application integration using actual experimental and testing evidence.

The process connects the research problem to the developed software artefact and provides a basis for evaluating whether the implemented component fulfils its intended purpose.

Research phase	**Main activity**	**Related objective**
Problem identification	Analyse yield-estimation and disease-impact challenges.	Overall objective
Data and feature preparation	Identify relevant environmental, soil and crop variables.	SO1, SO2
Model integration	Use the baseline yield and disease-loss model artefacts.	SO1, SO2
Scenario development	Calculate untreated and treated yield estimates.	SO3
Application integration	Connect the prediction endpoint with the Flutter interface.	SO4
Evaluation	Assess prediction quality and software functionality.	SO5

Table 2.1: Mapping of Research Phases to Objectives

2.2 Overview of the Proposed System

The individual component forms part of the wider Smart Agriculture Decision Support System developed under Project R26-IT-066. Its purpose is to transform available field conditions and disease-related information into harvest-oriented predictions and scenario-based decision support.

The component uses two predictive stages. First, a baseline yield model estimates expected paddy yield using environmental, soil and crop-related inputs. Second, a disease-loss model estimates the percentage of yield loss associated with the supplied disease information. These outputs are combined to calculate untreated and treated yield scenarios.

The current implementation uses Python-based prediction logic, a Flask backend and a Flutter application interface. Firebase integration supports access to available field telemetry. The wider project also contains separate components responsible for field monitoring, disease analysis and crop or market-related information. These are treated as supporting components rather than as the individual implementation contribution of this research.

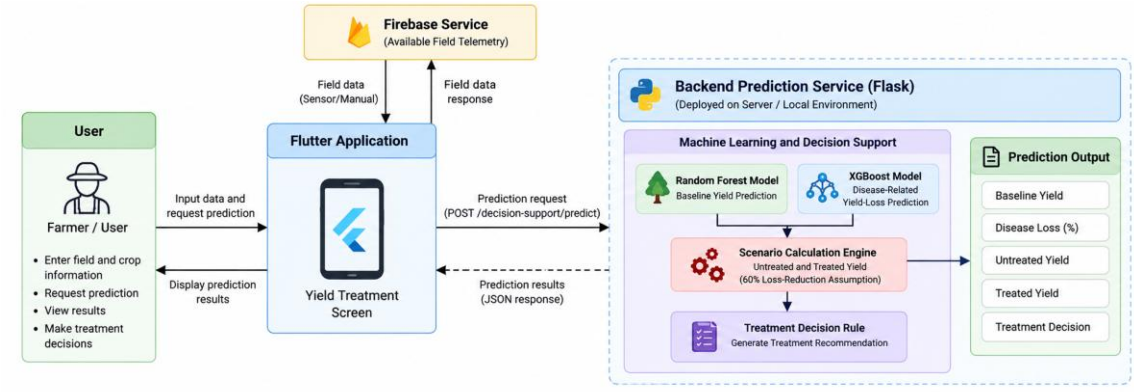


Figure 2.1:System overview diagram

2.3 Data Inputs and Sources

The prediction component requires information about field conditions, soil nutrients, crop characteristics and disease status. The implementation constructs its prediction inputs from the fields received by the decision-support service. Available field telemetry can also be retrieved through the Firebase service used by the Flutter application.

Input feature	**Role in the prediction process**
Soil moisture	Represents the moisture condition of the field.
Temperature	Represents the environmental temperature.
Humidity	Represents the surrounding humidity condition.
Nitrogen	Represents an important soil nutrient.
Phosphorus	Represents an important soil nutrient.
Potassium	Represents an important soil nutrient.
Growth stage	Provides information about the crop's development stage.
Season	Represents the cultivation season, such as Maha or Yala where supported.
Paddy variety	Identifies the selected variety where supported by the model.
Disease type	Provides the disease category for disease-loss estimation.
Disease severity	Represents the supplied severity value used by the disease-loss model.

Table2.2: Input Features and Their roles

The research proposal identifies BG 300 and AT 362 as target varieties and Maha and Yala as relevant cultivation seasons. However, the actual values supported by the deployed model must follow the feature encoding and categories used in the implementation.

The final thesis should identify the exact training-data sources, dataset sizes, target variables and train-test split from the actual model-training records. Proposal-level dataset quantities should not be presented as the final dataset statistics unless they are verified.

2.4 Data Preprocessing and Feature Preparation

Data preparation is important because the prediction models require inputs in a consistent structure. The implemented service constructs feature data frames from the received request values and aligns the features with the columns expected by the trained models.

Categorical variables such as growth stage, season, paddy variety and disease type require representations compatible with the trained model. The inspected implementation uses indicator columns for relevant categories. Disease severity is retained as a numerical input in the disease-loss prediction stage.

The main preparation activities are:

- **Input validation:** Check that the required prediction fields are supplied in an acceptable format.
- **Feature construction:** Arrange the received values into the data structure expected by each model.
- **Categorical encoding:** Represent supported categorical values using the feature columns expected by the trained artefacts.

- **Feature alignment:** Maintain the correct feature names and ordering for model inference.
- **Output preparation:** Convert model predictions into values that can be used by the scenario-calculation stage and returned by the API.

The exact missing-value treatment, outlier handling, normalization procedures and other training-time preprocessing operations must be confirmed from the actual training notebook or pipeline. They should not be described as completed procedures unless implementation evidence confirms them.

2.5 Baseline Paddy Yield Estimation

The first predictive stage estimates the baseline paddy yield before applying the predicted disease-related loss. The implemented decision-support service loads the trained Random Forest model artefact named random_forest_model2.pkl.

The model receives the supported environmental, soil and crop-related features and returns a baseline yield estimate. This output represents the estimated crop yield under the supplied conditions before the disease-loss adjustment used by the scenario engine.

The general prediction process can be represented as:

[
Y_{\text{base}}=f_{\text{RF}}(X_{\text{field}},X_{\text{crop}})
]

where:

- (Y_{\text{base}}) is the estimated baseline yield.
- (f_{\text{RF}}) represents the trained Random Forest model.
- (X_{\text{field}}) represents the supported field and environmental features.

- (X_{\text{crop}}) represents the supported crop-related features.

The equation describes the model's role rather than its internal training procedure. The training algorithm settings, dataset split and selected hyperparameters should be reported from the actual experiment records.

The proposal identifies regression evaluation as an important part of the research. Therefore, the baseline model should be evaluated against actual reference values using appropriate metrics, including the coefficient of determination ((R^2)), Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), where the required test data are available.

2.6 Disease-Related Yield-Loss Estimation

The second predictive stage estimates the percentage of yield loss associated with the supplied disease conditions. The inspected implementation loads the trained XGBoost artefact named model2_xgboost.pkl.

The model uses the supported disease type, disease severity and crop-related features to generate a predicted loss percentage. This prediction is then used by the scenario engine to estimate the untreated yield.

The calculation can be expressed as:

[
L=f_{\text{XGB}}(X_{\text{disease}},X_{\text{crop}})
]

where:

- (L) is the predicted disease-related yield-loss percentage.
- (f_{\text{XGB}}) represents the trained XGBoost model.
- (X_{\text{disease}}) represents the supported disease-related inputs.

- X_{crop} represents the relevant crop-context inputs.

The research proposal originally described disease-loss modelling using reference information and disease-related variables. However, the model artefact used by the current prediction service is XGBoost. The methodology therefore documents the verified implementation and does not describe the proposed polynomial-regression approach as the deployed method.

The final evaluation should compare predicted loss values with appropriate reference or labelled values. Any claim about model accuracy or its suitability for particular disease types must be supported by the actual validation results.

2.7 Yield Scenario Calculation

The scenario engine combines the outputs from the baseline yield and disease-loss models to estimate possible harvest outcomes. The current implementation calculates the untreated yield and treated yield using the predicted loss percentage.

2.7.1 Untreated Yield

The untreated yield is calculated by applying the predicted disease-related loss to the baseline yield.

$$Y_{\text{untreated}}= Y_{\text{base}}\left(1-\frac{L}{100}\right)$$

where L is the predicted loss percentage.

2.7.2 Treated Yield

The current implementation assumes that treatment reduces the predicted disease-related loss by 60%. The remaining loss is therefore calculated as:

$$L_{\text{treated}}=0.4L$$

The treated yield is then calculated as:

$$Y_{\text{treated}}=Y_{\text{base}}\left(1-\frac{L_{\text{treated}}}{100}\right)$$

This calculation is an implementation assumption. It does not establish that a real treatment will reduce disease-related losses by 60%. The assumption requires appropriate agricultural evidence or field validation before it can be interpreted as a reliable treatment-effect estimate.

2.7.3 Treatment-Related Decision Output

The current decision rule compares the estimated treated yield with the estimated untreated yield. If the treated yield is greater, the service returns a treatment-related recommendation; otherwise, it returns a decision not to treat.

This rule provides a simple comparison of predicted yield outcomes. However, it does not by itself establish whether treatment is financially optimal because treatment costs, crop prices, uncertainty and other practical factors must also be considered. Financial benefit calculations should only be claimed if the relevant inputs and calculations are implemented and verified.

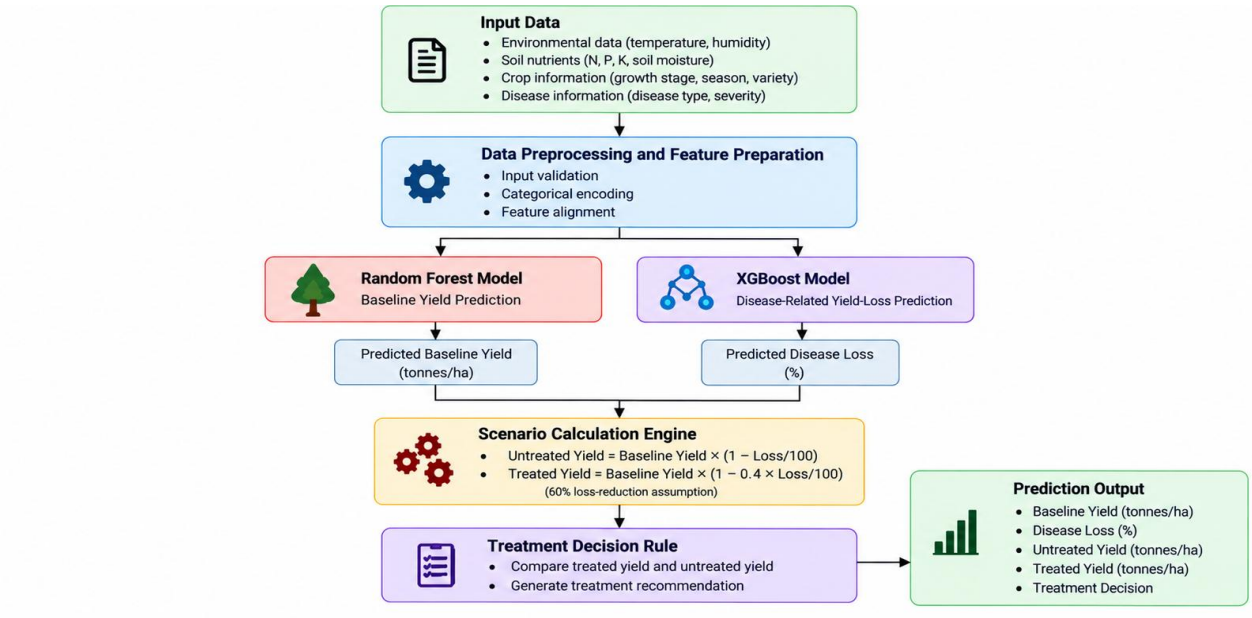


Figure 2.0.2:System overview diagram

2.8 Software Architecture and Implementation

The component uses a client-server architecture in which the Flutter interface communicates with a Python backend to obtain yield-related predictions. The backend performs model inference and scenario calculations, while the interface presents the returned results.

2.8.1 Backend Prediction Service

The inspected implementation contains a Flask-based prediction service with the endpoint POST /decision-support/predict. The endpoint receives the supported input fields, constructs the model features, executes the prediction workflow and returns the resulting yield and decision-support outputs.

The returned output fields include:

- base_yield
- loss_percent
- untreated_yield
- treated_yield
- decision

The final thesis should include an example request and response based on the actual endpoint implementation. Any claims about authentication, production deployment, error handling or security controls should be supported by evidence from the code and configuration.

2.8.2 Flutter Application Interface

The Flutter application includes a yield-treatment interface through which the user can access the component's decision-support functionality. The interface is responsible for

presenting the available inputs and the returned prediction information in a form that can be interpreted by the user.

The final report should include screenshots of the running application showing the relevant input screen and prediction results. Screenshots must be captured from the actual implementation.

2.8.3 Firebase Integration

The project includes a Firebase service in the Flutter application that can retrieve available field telemetry. This supports the use of field-related information within the wider application workflow.

Firebase integration should not be interpreted as evidence that every sensor input is continuously available or that the complete system has been deployed in a live paddy field. The final report should describe only the data retrieval and integration behaviour verified through the application and available testing evidence.

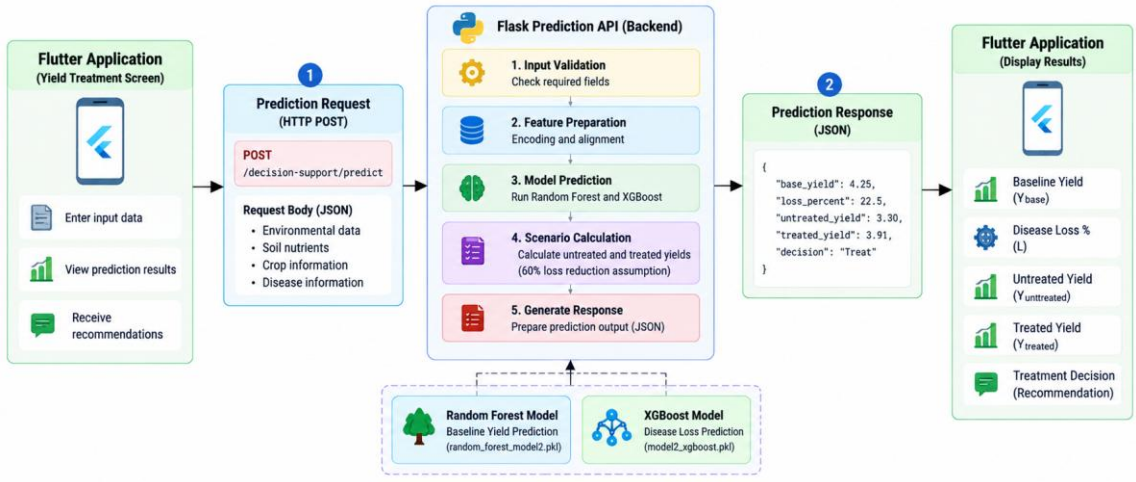


Figure 2.0.3:Implemented IT22266446 Component Architecture

The diagram should show the Flutter yield-treatment screen, Firebase telemetry service, Flask prediction endpoint, the Random Forest and XGBoost model artefacts, scenario calculations and the returned prediction outputs. Use the actual implemented architecture rather than the originally proposed React.js and FastAPI design.

2.9 Functional and Non-Functional Requirements

The requirements provide a basis for verifying whether the component performs its intended tasks.

ID	Requirement
FR1	The component shall accept the supported field, soil and crop-related input features.
FR2	The component shall generate a baseline yield estimate using the implemented Random Forest model.
FR3	The component shall estimate disease-related yield loss using the implemented XGBoost model.
FR4	The component shall calculate untreated and treated yield scenarios using the implemented equations.
FR5	The component shall generate the configured treatment-related decision output.
FR6	The prediction service shall return the required output fields through its API endpoint.
FR7	The application shall present the available prediction results through the Flutter interface.
FR8	The application shall retrieve available field telemetry through the configured Firebase service.

Table 2.3: Functional Requirements

Area	Requirement
Reliability	Predictions and scenario calculations should be consistent for equivalent valid inputs.
Performance	Prediction response time should be measured using repeatable tests.
Usability	The interface should present yield and loss information clearly.
Maintainability	The prediction service, model artefacts and interface should remain identifiable as separate implementation elements.
Data integrity	Input features should match the structure expected by the trained models.
Security	Data access and backend configuration should be reviewed and tested before making security claims.

Table 2.4: Non-Functional Requirements

These requirements establish what should be evaluated. Their inclusion in this table does not mean that every requirement has already passed testing.

2.10 Testing and Implementation

Testing is required to verify the behaviour of the prediction models, scenario calculations, API and application interface. The evaluation should distinguish between model-level testing and software functional testing.

2.10.1 Model Evaluation

The baseline-yield model and disease-loss model should be evaluated separately using appropriate reference values. The following regression metrics may be used where the actual test data are available.

[MAE=\frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|]

[RMSE=\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}]

The (R^2) metric may also be reported to describe the proportion of variance explained by the model on the evaluation data. The final results should identify the evaluation dataset and report the actual measured values. Proposal targets must not be substituted for measured results.

2.10.2 Scenario Calculation Testing

Scenario testing should verify whether the calculations follow the implemented equations. For a valid positive loss percentage within the intended input range, the untreated yield should be lower than the baseline yield. Under the current 60% loss-reduction assumption, the calculated treated yield should be greater than the untreated yield.

These checks verify mathematical consistency with the implemented rules; they do not demonstrate that the assumed treatment effect occurs in real agricultural conditions.

2.10.3 API and Integration Testing

The prediction endpoint should be tested using valid inputs and appropriate negative test cases. Testing should verify that:

- A valid request produces the expected output fields.
- Model predictions are returned in the required format.
- The scenario calculations produce consistent outputs.
- Missing or invalid inputs are handled appropriately.
- The Flutter interface can obtain and display the prediction results.
- Firebase data retrieval behaves as expected for the available test configuration.

2.11 Commercialization Aspects of the Product

The component has potential to form part of a digital decision-support service for Sri Lankan paddy farmers. Its possible value lies in presenting estimated baseline yield, disease-related loss and alternative yield scenarios through a single application workflow.

Potential users include paddy farmers and agricultural advisers who need to interpret crop-condition information alongside expected harvest outcomes. The wider R26-IT-066 platform could provide complementary field, disease and market information, subject to successful integration and validation.

Potential commercialization considerations include:

- **Target users:** Smallholder and medium-scale paddy farmers and relevant agricultural advisers.
- **Delivery method:** An application-based service connected to a backend prediction service.
- **Potential value:** More accessible presentation of estimated yield and disease-related risk.
- **Operational requirements:** Reliable data inputs, model maintenance, suitable hosting, technical support and appropriate data protection.
- **Validation requirements:** Evidence of model performance, field relevance and user acceptance before commercial use.

A possible future business model could involve partnerships with agricultural service providers, institutions or farming organizations. However, pricing, customer demand, revenue forecasts and commercial viability require separate evidence. The component should not be presented as a validated commercial product solely because its prediction workflow has been implemented.

2.12 Methodological Limitations

Several limitations must be considered when interpreting the output of this component.

First, the reliability of the predictions depends on the quality, representativeness and consistency of the training data. Second, the final model performance must be established through documented evaluation rather than inferred from the existence of trained model artefacts. Third, the treated-yield calculation uses a fixed 60% reduction in predicted disease loss, which has not been established as a field-validated treatment effect. Fourth, the current decision rule compares predicted yields rather than a verified net financial benefit. Finally, the quality of the complete workflow depends on the availability and correct integration of the relevant field and disease information.

These limitations should be considered in the results and discussion chapter and addressed in future work.

2.13 Chapter Summary

This chapter described the research approach and the methodology used to develop the disease-aware paddy harvest yield estimation component. It covered the input features, model roles, scenario calculations, software architecture, functional requirements, evaluation strategy and commercialization considerations. It also distinguished the implemented prediction workflow from proposed features and assumptions that require further validation. The next chapter should present the actual experimental results, software testing evidence, research findings and discussion of the extent to which the objectives have been achieved.

3. RESULTS AND DISCUSSION

This chapter reports the results that can be supported by the inspected implementation and the evidence supplied with the research materials. The most important distinction is between an implemented artifact and an experimentally validated result. The source code and model files provide strong evidence that the prediction pipeline was implemented. However, the supplied evidence pack does not contain a final experiment log containing the training-set size, held-out test metrics, measured API latency or completed farmer usability scores. Accordingly, this chapter reports verified artifact characteristics and provides the exact structure required for the final numerical evaluation without converting proposal targets into fabricated results.

The dataset supplied for baseline-yield evaluation contains 12,437 records and ten columns: nine predictors (soil_moisture, temperature, humidity, nitrogen, phosphorus, potassium, growth_stage, season and variety) and the yield target. The notebook output reports no null values, and an independent exact-duplicate check found no duplicate rows. However, 6,300 target values equal 2800.00. The source and meaning of this repeated target value have not yet been established and should be investigated before drawing conclusions about generalisation to real-world field conditions.

3.1 Implemented Artifact Results

The first implementation result is the presence of a dedicated decision-support backend module. The module loads random_forest_model2.pkl and model2_xgboost.pkl, constructs two feature matrices and produces five principal outputs. This confirms that the individual component has been separated into a deployable analytical service rather than remaining only at proposal or notebook level.

The second implementation result is the presence of a dedicated Flutter yield-treatment screen and an API service method named predictYieldDecision. The client is therefore designed to consume the analytical service through a reusable API abstraction. The screen

also integrates with the Firebase service, creating a path for recent telemetry to populate nutrient inputs.

A screenshot captured from the user's running Flutter application is included below as interface evidence. It displays Base Yield = 3481.12 kg/ha, Disease Loss = 0.00%, Untreated Yield = 2489.69 kg/ha, Treated Yield = 3084.55 kg/ha and Decision = TREAT.

These values are internally inconsistent with the documented scenario equation because a zero loss would ordinarily make untreated yield equal to baseline yield. The screenshot therefore evidences that the interface renders output fields, not that this particular prediction is correct. The user has elected to defer fixing the Predict Yield & Decision issue until after this draft is reviewed.

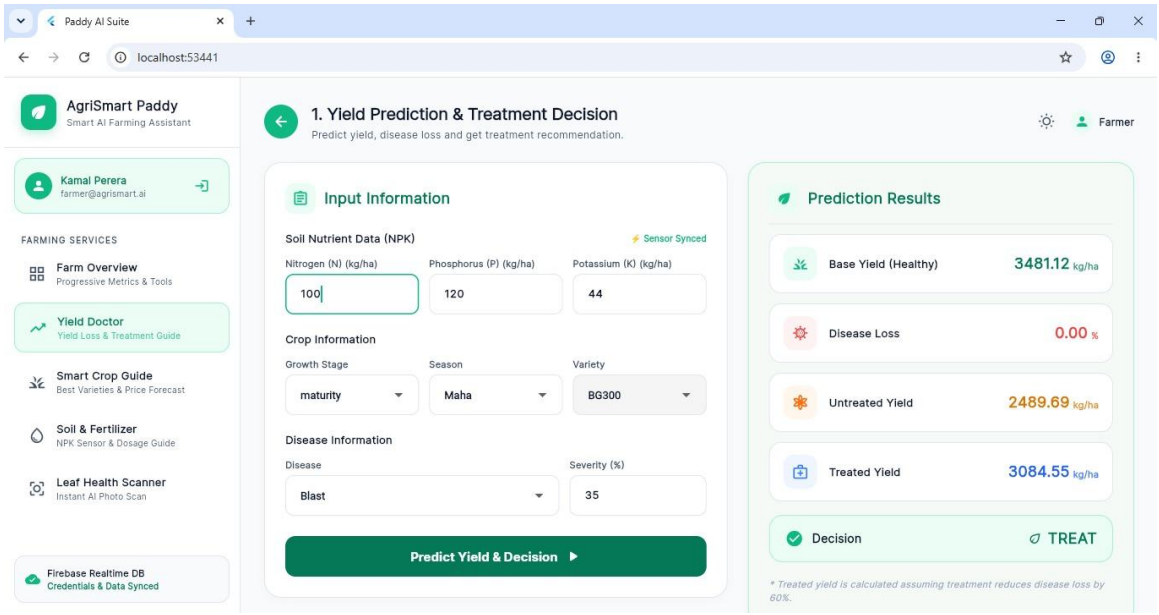


Figure 3.0: Captured flutter yield-treatment interface showing the current prediction output

3.2 Base-Yield Model Artifact Analysis

Inspection of random_forest_model2.pkl shows a RandomForestRegressor with 100 trees. The model bundle stores the feature-column list alongside the trained estimator. The 15 stored columns are soil_moisture, temperature, humidity, nitrogen, phosphorus, potassium, five growth-stage indicators, two season indicators and two variety indicators. This feature representation is consistent with the source-code construction function and demonstrates that categorical variables were encoded before inference.

Exploratory Dataset Visualization

Before interpreting the model results, the target distribution, potential outliers and relationships among the available numerical and encoded features were examined. Figures 3.2–3.4 provide visual evidence from the supplied Colab analysis. These plots are descriptive and do not independently establish model accuracy.

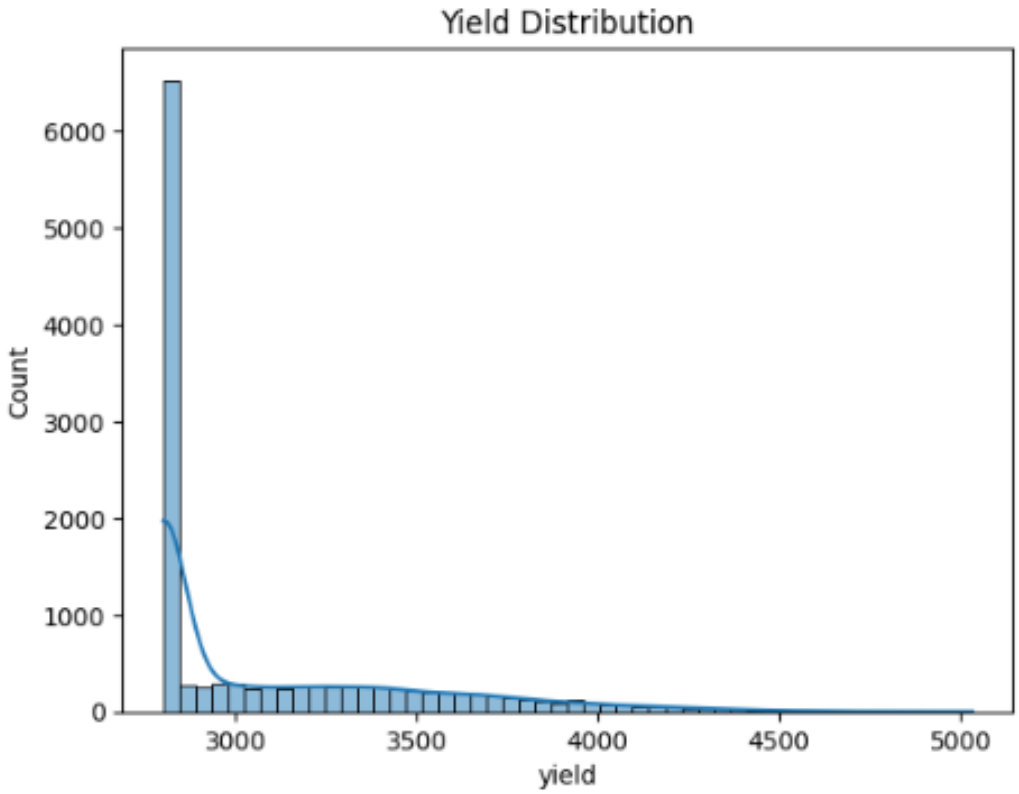


Figure 3.0.1:Distribution of Paddy Yield Values in the Dataset

The histogram shows a strong concentration of observations near the lower end of the displayed yield range and a tail extending toward larger values. The repeated concentration around approximately 2,800 should be checked against the raw records before drawing conclusions about the dataset's representativeness.

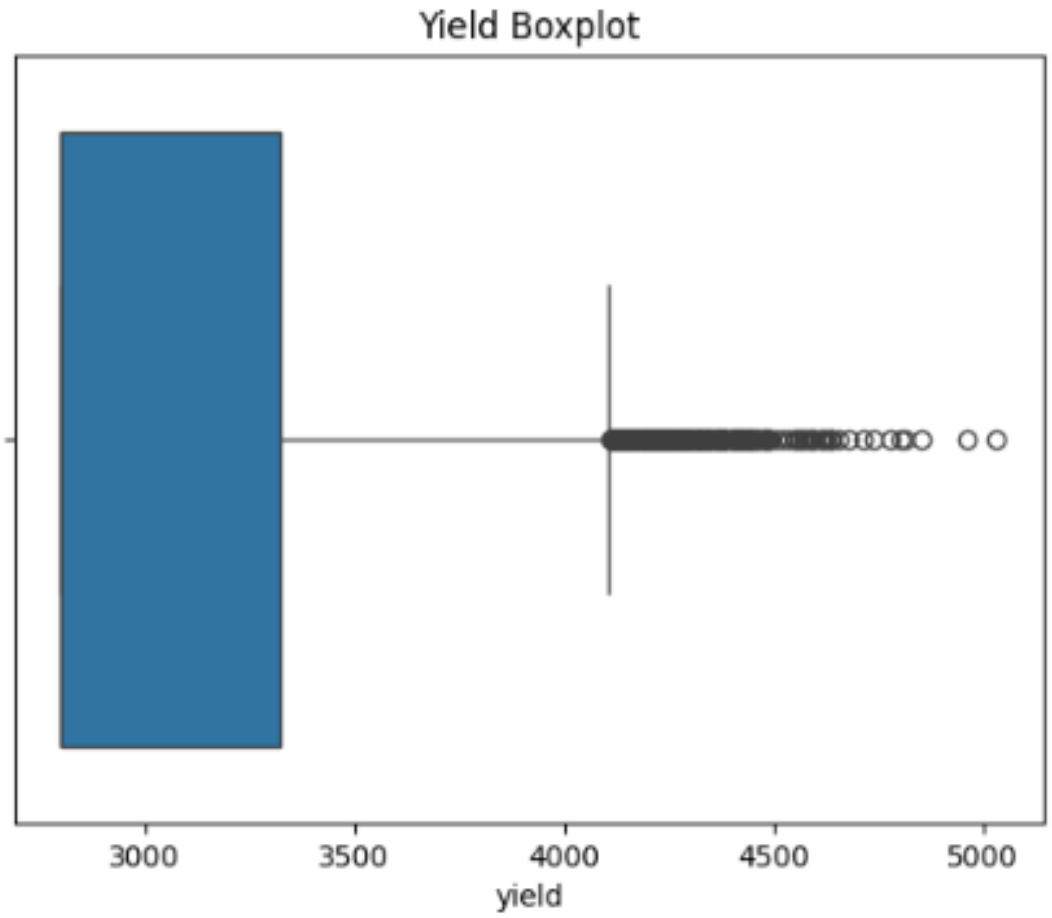


Figure 3.0.2: Boxplot of Paddy Yield Values

The boxplot shows multiple observations beyond the upper whisker. These are potential outliers under the boxplot rule, not automatically data errors; each should be interpreted against the source records and agricultural context before any removal or transformation.

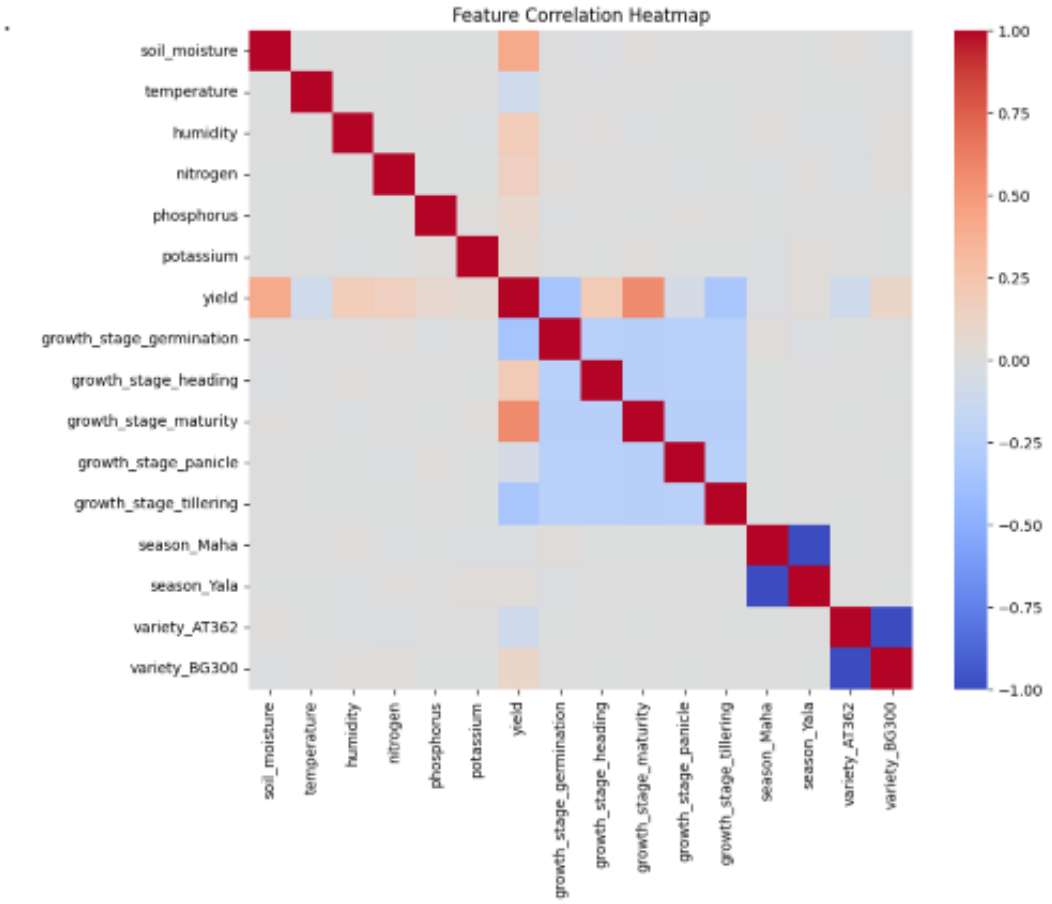


Figure 3.0.3: Correlation Heatmap of Yield and Input Features

The heatmap visualises pairwise correlation patterns between yield and the selected input features. It provides exploratory evidence about linear relationships, but correlation does not demonstrate causation. Exact coefficient values should be taken from the numeric correlation matrix rather than estimated from colours.

In addition to artifact inspection, the uploaded Decision_Support.ipynb reports baseline-yield regression metrics from an 80/20 train–test split with random_state=42. The test partition contains 2,488 records from the 12,437-row dataset. Table 3.1A records the values reported by that notebook. The notebook reports R^2 and MAE but does not report RMSE; no RMSE value is inferred here. The target unit should be verified against the dataset documentation before final submission.

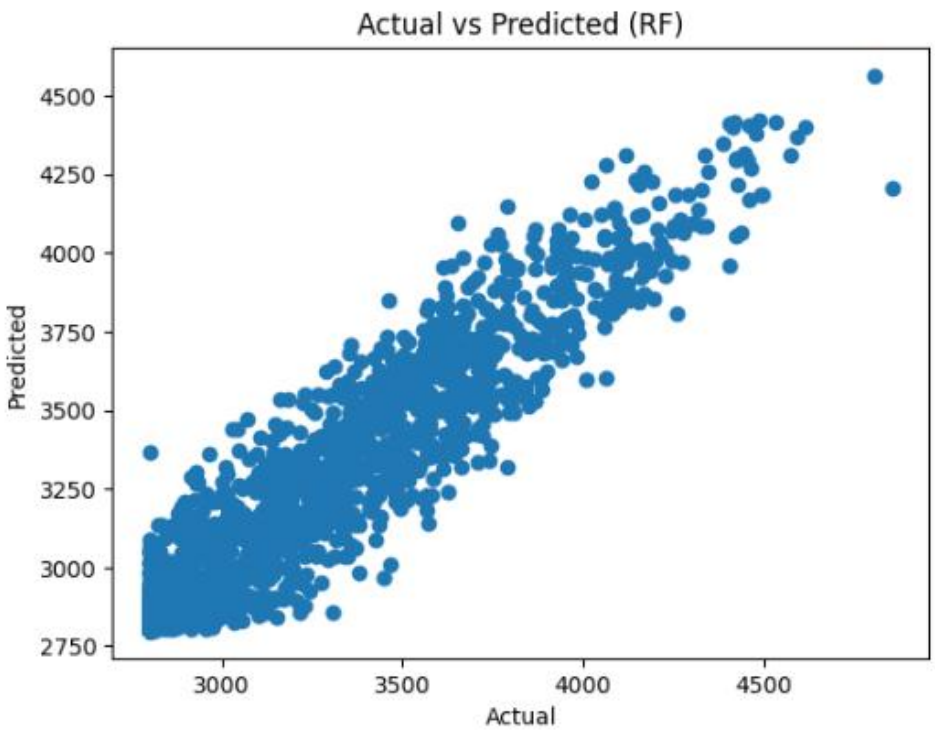


Figure 3.0.4: Actual vs. Predicted Paddy Yield Using the Random Forest Model

The scatter plot shows a general positive relationship between actual and predicted yield values, although the points do not align perfectly and some prediction differences are visible. This visual pattern is an initial diagnostic only; predictive performance should be interpreted using the reported held-out metrics, including R^2, MAE and RMSE.

The model's internal feature-importance values provide an implementation-level diagnostic. The largest stored importance values are associated with growth_stage_maturity and soil_moisture, followed by growth_stage_heading and humidity. These values show which features the fitted Random Forest used most strongly in its split structure. They should not be interpreted as causal agronomic relationships and should not replace formal model evaluation.

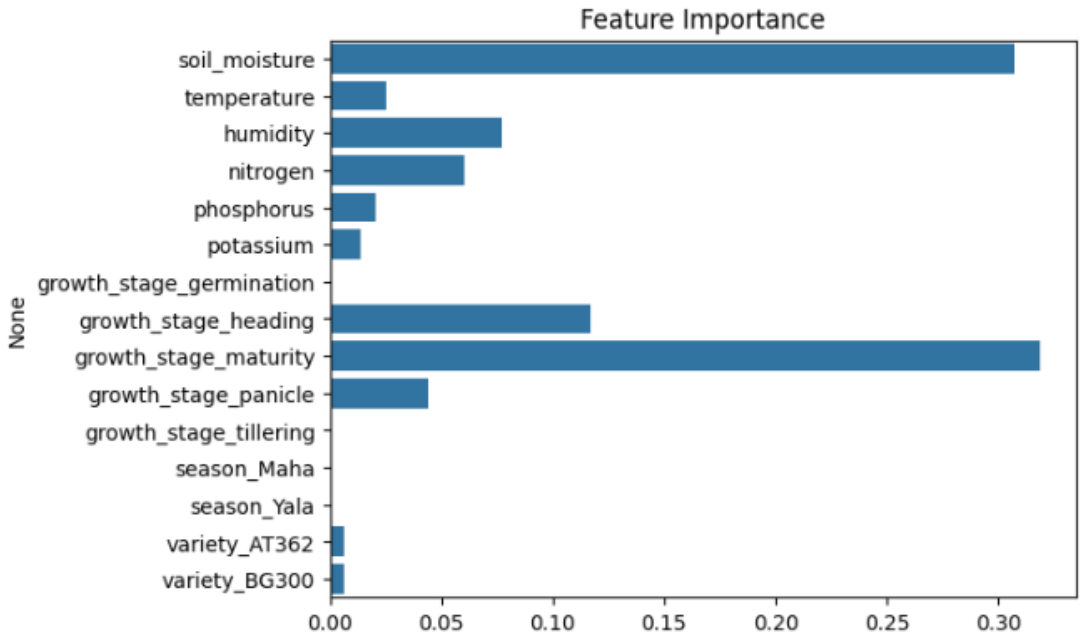


Figure 3.0.5: Feature Importance of the Random Forest Yield Prediction Model

In this plot, soil moisture and the maturity growth-stage indicator have the largest displayed importance values, followed by other features such as heading stage, humidity and nitrogen. Feature importance describes the model's relative reliance on inputs in this dataset; it does not prove that a feature causes a change in yield.

3.3 Disease-Loss Model Artifact Analysis

The disease-loss estimation component uses an XGBoost regression model stored as model2_xgboost.pkl. It predicts the percentage of potential paddy yield loss based on disease severity, disease type and crop growth-stage features. The predicted loss is subsequently used in the yield scenario calculations.

The feature-importance analysis identifies the grain-stage indicator, disease severity, tillering-stage indicator and Blast indicator among the more important features. BrownSpot has zero stored importance in the inspected model. However, this does not mean that Brown Spot has no actual effect on paddy yield.

The model's predictive accuracy must be evaluated separately using appropriate metrics such as (R^2), MAE and RMSE. Actual evaluation values should be included only when supported by the training or testing results.

3.4 Scenario Calculation Verification

The scenario equations were verified directly from the backend source. For a predicted loss L greater than zero, untreated yield is lower than base yield. The treated scenario uses 0.4L, so the treated yield is mathematically higher than the untreated yield whenever the base yield is positive and L is positive. Consequently, the current decision rule will normally select TREAT for positive predicted loss. This is an important finding because it reveals that the current decision engine is a deterministic comparison rather than a full economic optimization.

A genuine treatment decision should compare expected recovered yield value against treatment cost and uncertainty. If P denotes expected paddy price per kilogram, Ct treatment cost and E the expected treatment efficacy, an economic formulation could compare P(Yt - Yu)E with Ct. The current implementation does not perform this calculation inside the decision-support route. Therefore, the thesis does not claim that the current TREAT/DO NOT TREAT output is a validated profitability decision. It is a yield-scenario decision based on the implemented loss-reduction assumption.

3.5 API Verification

The component uses a Flask-based REST API with the endpoint POST /decision-support/predict. The backend accepts eleven required fields: soil moisture, temperature, humidity, nitrogen, phosphorus, potassium, growth stage, season, variety, disease and severity.

For a valid request, the API is designed to return five outputs: base_yield, loss_percent, untreated_yield, treated_yield and decision. API verification should check valid requests,

missing fields, unsupported categorical values and the consistency of the returned scenario calculations.

Source-code inspection identified a possible integration issue because the inspected Flutter request payload does not explicitly include soil moisture, temperature or humidity, although the backend requires these fields. This mismatch must be corrected or confirmed through testing before successful integration can be claimed.

The final evaluation should include actual API request and response evidence, test outcomes and measured response times where available. Source-code inspection confirms the intended API structure but does not independently prove successful end-to-end execution.

3.6 Functional Test Design

Table 6 presents the minimum functional test suite for the component. TC01 checks a complete valid request. TC02 checks each missing required field. TC03 checks a valid disease type. TC04 checks a severity boundary. TC05 checks an invalid categorical value. TC06 verifies that base yield is returned as a numeric value. TC07 verifies that untreated yield is computed from the returned base yield and loss. TC08 verifies the treated-yield equation. TC09 verifies the decision rule. TC10 checks client display of all response fields. The final submitted thesis should include pass/fail evidence and screenshots for each applicable test.

ID	Scenario	Input / Action	Expected Result	Evidence Status
TC01	Complete valid request	All required fields supplied	Five prediction outputs returned	Runtime evidence required

TC02	Missing soil moisture	Remove soil_moisture	HTTP 400 with missing-field error	Source logic verified
TC03	Missing temperature	Remove temperature	HTTP 400 with missing-field error	Source logic verified
TC04	Missing humidity	Remove humidity	HTTP 400 with missing-field error	Source logic verified
TC05	Missing disease severity	Remove severity	HTTP 400 with missing-field error	Source logic verified
TC06	Scenario arithmetic	Compare returned values to equations	Untreated and treated values follow formulas	Source logic verified
TC07	Positive loss decision	Use positive predicted loss	TREAT normally selected under current rule	Source logic verified
TC08	Flutter submission	Submit form from client	Request reaches endpoint and response is shown	Runtime evidence required
TC09	Telemetry population	Load Firebase telemetry	N/P/K fields populated when values exist	Source integration verified

TC10	Invalid categorical input	Use unsupported category	System rejects or safely handles input	Runtime evidence required

Table 3.1: Functional and integration test cases

3.7 Non-Functional Evaluation

The proposal defines a response-time target of three seconds and a usability target of at least 80% task completion among ten farmers. These targets are useful for the evaluation framework, but the supplied evidence does not contain measured latency logs or a completed user-study dataset. The correct reporting approach is therefore to leave the measured-result cells open until the actual logs or survey data are inserted. A proposal target must never be rewritten as an achieved result.

Evaluation item	**Target / criterion**	**Actual result**	**Interpretation**
Base-yield R^2	≥0.85	Pending	Proposal target only
Disease-loss R^2	≥0.80	Pending	Proposal target only
Base-yield MAE	Report	Pending	Final experiment required
Base-yield RMSE	Report	Pending	Final experiment required
Disease-loss MAE/RMSE	Report	Pending	Final experiment required
API response time	≤3 s	Not measured	Runtime measurement required
Usability	≥80% task completion		User-study evidence required

Integration	Successful client/API flow		Runtime evidence required

Table 3.2: Evidence-based evaluation summary

3.8 Research Findings

This section presents and discusses the main findings of the Disease-Aware Paddy Harvest Yield Estimation and Treatment Decision Support component (IT22266446). The discussion covers dataset characteristics, baseline-yield prediction, disease-loss estimation, scenario calculations and application integration. It distinguishes findings supported by the available experimental results from those confirmed only through source-code inspection or still requiring runtime verification.

3.8.1 Findings from Dataset Analysis

The dataset used for baseline-yield evaluation contains 12,437 records and ten columns, comprising nine input features and one yield target. The input features include soil moisture, temperature, humidity, nitrogen, phosphorus, potassium, growth stage, season and paddy variety. The supplied notebook reports no missing values, and an independent check found no exact duplicate rows.

However, 6,300 records contain a yield value of 2800.00. This repeated value represents a substantial concentration within the target distribution and requires further investigation. Its origin has not been established from the available evidence. It could reflect a genuine characteristic of the dataset or a data-generation or preparation procedure; therefore, it should not automatically be interpreted as either a valid agricultural pattern or an error.

Figures 3.2 and 3.3 present the yield distribution and boxplot, respectively. These visualisations show a concentration of values near the lower end of the displayed range and a number of higher-value observations. The boxplot identifies potential outliers, but the presence of an outlier does not necessarily mean that the observation is incorrect. Such

values should be assessed using the dataset documentation and source records before being removed or modified.

Figure 3.4 presents the correlation heatmap for the yield target and selected input features. It provides an exploratory view of the relationships between the variables. Nevertheless, correlation does not establish causation, and the strength of any specific relationship should be interpreted using the underlying numerical correlation values rather than colour intensity alone.

Overall, the dataset analysis provides useful information about the structure and distribution of the available data. The repeated target value and the presence of potential outliers highlight the importance of investigating data provenance and representativeness before applying the model to new agricultural conditions.

3.8.2 Findings from Baseline-Yield Prediction

The implemented baseline-yield prediction component uses a Random Forest regression model stored in random_forest_model2.pkl. Inspection of the model artifact identified a Random Forest regressor containing 100 trees. Its stored feature-column list contains 15 columns, including environmental variables, soil nutrients and encoded growth-stage, season and variety indicators.

The supplied Decision_Support.ipynb reports baseline-yield evaluation results using an 80/20 train–test split with random_state=42. The test partition contains 2,488 records. The reported comparison includes Random Forest, XGBoost and Linear Regression models.

Model	**(R^2)**	**Mean Absolute Error (MAE)**
Random Forest	0.9161	73.4005
XGBoost	0.9155	80.2946
Linear Regression	0.7524	164.1272

Figure 3.3: Baseline-Yield Model Comparison

Source: Results reported in the supplied Decision_Support.ipynb. The target unit should be confirmed against the dataset documentation.

The reported results indicate that Random Forest achieved the highest (R^2) and lowest MAE among these three models in the evaluated baseline-yield comparison. Its (R^2) value of 0.9161 indicates that the model explains approximately 91.61% of the variation in the target values within that evaluation setting. Its MAE of 73.4005 represents the mean absolute difference between the predicted and actual target values, expressed in the target's measurement unit.

XGBoost achieved a similar (R^2) value of 0.9155, with an MAE of 80.2946. Linear Regression produced a lower (R^2) of 0.7524 and a higher MAE of 164.1272. These results support the selection of Random Forest for the implemented baseline-yield prediction stage within the reported comparison.

Figure 3.5 presents the actual-versus-predicted values for the Random Forest model. The overall positive relationship between actual and predicted values visually supports the numerical comparison. However, the spread of individual points indicates that prediction errors remain. Therefore, the graph should be interpreted together with the numerical metrics rather than as independent proof of perfect prediction.

Figure 3.5 presents the feature-importance values of the Random Forest model. Soil moisture and the maturity growth-stage indicator have comparatively high importance values in the displayed plot. Other features, including heading stage, humidity and nitrogen, also contribute according to the plotted values. These importance scores describe the fitted model's use of the input features; they do not prove that the corresponding variables independently cause changes in paddy yield.

The reported metrics apply to the baseline-yield evaluation only. They must not be used as evidence of the accuracy of the separate disease-loss model. In addition, the high concentration of repeated yield values should be investigated when assessing how well the model may generalise to independent field observations.

3.8.3 Findings from Disease-Loss Estimation

The disease-loss estimation component uses the XGBoost regression artifact stored in model2_xgboost.pkl. The backend constructs an eight-feature input matrix for this model. Its intended role is to estimate the percentage of potential yield loss associated with the supplied disease conditions and crop growth stage.

The stored feature-importance analysis identifies the grain-stage indicator, disease severity, tillering-stage indicator and Blast indicator among the more important features in the inspected model. The BrownSpot feature has a stored importance value of zero in that analysis. This indicates that the feature receives no importance under the inspected model's stored importance measure; it does not establish that Brown Spot has no effect on paddy yield in real agricultural conditions.

The disease-loss prediction is important because its output is used in the subsequent untreated- and treated-yield calculations. Errors in this estimate may therefore affect both scenario outputs and the resulting treatment-related decision.

However, the available evidence does not establish final held-out evaluation metrics for the disease-loss model. The baseline-yield results reported in Section 3.8.2 cannot be transferred to this model because the two models predict different target variables. Consequently, the disease-loss model can be discussed as an implemented component, but its predictive accuracy and generalisation performance remain to be established using appropriate labelled or reference loss values and a documented evaluation dataset.

3.8.4 Findings from Scenario Calculation and Treatment Decision Support

The scenario calculation engine combines the baseline yield predicted by Random Forest with the disease-related loss estimated by XGBoost. The implemented workflow calculates an untreated-yield estimate by applying the predicted loss percentage to the baseline yield.

The treated-yield scenario uses a fixed assumption that treatment reduces the predicted loss by 60%. The remaining loss is therefore represented by 40% of the original predicted

loss. The system then compares the estimated treated and untreated yields to produce a treatment-related decision output.

This approach provides an interpretable way of representing the potential effect of disease-related loss on expected harvest yield. It also separates the baseline-yield estimate from the disease-loss adjustment, allowing the scenario calculations to use the outputs of the two prediction components.

Nevertheless, the 60% loss-reduction factor is a software assumption rather than a measured treatment effect. It has not been established as the effectiveness of a particular agricultural intervention through field experiments or other supporting evidence. Accordingly, the treated-yield value represents a hypothetical scenario under the current calculation rule.

The current decision rule also compares the estimated yields without explicitly including treatment cost, expected paddy price or uncertainty in the predicted treatment effect. A higher estimated treated yield therefore does not necessarily mean that treatment is financially optimal. The implementation provides a basic yield-scenario comparison, but it should not be interpreted as a fully validated financial optimisation model.

3.8.5 Findings from Application and API Integration

The implemented component includes a Flutter yield-treatment interface and a Flask-based backend prediction endpoint, POST /decision-support/predict. The backend is designed to accept field, soil, crop and disease inputs, invoke the prediction models, calculate the yield scenarios and return the principal outputs: base_yield, loss_percent, untreated_yield, treated_yield and decision.

The Flutter interface demonstrates the intended presentation layer for making prediction information accessible to users. The project also includes a Firebase service that provides a pathway for retrieving available telemetry within the application workflow. The existence of these components demonstrates implementation work, but it does not

independently establish that the complete client-server workflow has passed end-to-end testing.

The available interface screenshot displays a base yield of 3481.12 kg/ha, a disease-loss estimate of 0.00%, an untreated yield of 2489.69 kg/ha and a treated yield of 3084.55 kg/ha. These values are inconsistent with the documented scenario equations because a zero loss should ordinarily make the untreated yield equal to the baseline yield. Therefore, the screenshot demonstrates that the interface can display prediction fields, but the displayed values should not be treated as a verified, mathematically consistent prediction result.

Local backend execution also revealed an application initialisation error after the model artifacts had loaded successfully. Source-code inspection additionally identified a possible mismatch between environmental fields required by the backend and the request constructed by the Flutter client. These issues mean that successful end-to-end API integration cannot currently be claimed without further verification.

Consequently, the API and interface should be reported as implemented components whose complete runtime integration remains pending. A successful request and response, consistent scenario calculations and correct display of returned values are required before the complete workflow can be considered verified.

3.8.6 Overall Research Findings

The overall findings indicate that the IT22266446 component provides an implemented workflow for combining baseline-yield estimation, disease-related loss prediction and scenario-based yield comparison. The baseline-yield evaluation reports favorable results for Random Forest compared with the other models included in the supplied notebook. The stored model artifacts, feature-construction logic, scenario equations and application components provide evidence of the technical implementation.

At the same time, the results identify several limitations that affect the interpretation and practical use of the component. The repeated yield value in the dataset requires investigation, the disease-loss model needs its own documented evaluation, the treatment-effect assumption has not been validated, and the API integration requires successful runtime verification. The inconsistent interface output also demonstrates why visible application output should not automatically be treated as evidence of a correct prediction.

Therefore, the main contribution of this component is the implementation of a modular approach that connects yield estimation and disease-related loss to interpretable harvest scenarios. Further dataset investigation, model validation, treatment-effect calibration and system-level testing are necessary before the component can be relied upon for real-world agricultural or financial decisions.

3.9 Comparison with Existing Approaches

Compared with a disease-only classifier, the proposed component adds an explicit harvest-impact layer. Compared with a generic yield predictor, it introduces disease type and severity into the estimation pipeline. Compared with a static agricultural advisory screen, it exposes a model-backed scenario calculation through an API. Compared with a fully economic optimizer, however, the current implementation remains incomplete because treatment cost and market price are not used in the decision-support route itself. This balanced comparison is important for academic credibility and commercialization claims.

3.10 Discussion of Research Objectives

The findings provide different levels of evidence for the research objectives.

The objective of developing a baseline-yield prediction component is supported by the stored Random Forest artifact, its feature-construction logic and the baseline-yield metrics reported in the supplied notebook. Within the evaluated model comparison, Random Forest produced the highest (R^2) and lowest MAE among the three reported models.

The objective of estimating disease-related yield loss is supported at the implementation level by the stored XGBoost artifact and its disease-related feature structure. However,

the lack of documented final evaluation metrics means that its predictive accuracy cannot yet be confirmed.

The objective of comparing untreated and treated yield scenarios is supported by the implemented calculation equations and decision rule. Nevertheless, the fixed 60% loss-reduction assumption requires agricultural validation, and the current rule does not calculate verified net financial benefit.

The objective of integrating the analytical workflow into an application is supported by the presence of the Flask endpoint, Flutter interface and Firebase service. However, the initialization error, potential input-field mismatch and inconsistent interface output mean that successful end-to-end integration remains unverified.

Finally, the objective of evaluating the component is only partially supported by the available evidence. Baseline-yield metrics and source-code inspection findings are available, but disease-loss evaluation, complete runtime API testing, measured response time and farmer usability results require further evidence.

These findings distinguish the objectives supported by implementation evidence from those that require additional experimental or integration testing. Proposal targets should not be presented as achieved results unless the corresponding measurements are available.

Objective	**Target**	**Evidence**	**Status**	**Remaining evidence**
SO1	Base-yield estimation	RF artifact + feature construction	Implemented	Final metrics required
SO2	Disease-loss estimation	XGBoost artifact + severity/disease/stage inputs	Implemented	Final metrics/calibration evidence required
SO3	Scenario and	Scenario equations + decision rule	Implemented with fixed	Economic/treatment validation required

	treatment decision		efficacy assumption	
SO4	API + client integration	Flask route + Flutter screen + Firebase service	Implemented in source	End-to-end runtime evidence and field-contract correction required

Table 3.4: Objective achievement matrix

3.11 Limitations

The principal limitations are the absence of disease-loss model evaluation and complete system-level evaluation evidence in the supplied pack, the fixed treatment-loss reduction assumption, the absence of explicit treatment cost in the current decision route, and the identified client/API input mismatch. Additional limitations include model portability across package versions, dependence on upstream module availability, the need for larger Sri Lankan disease-yield datasets, and the need for independent farmer usability testing.

3.12 Practical Implications

The component demonstrates a practical pathway for converting disease observations into harvest-oriented information. For a farmer, the most meaningful output is an expected yield rather than only a disease label. For an extension officer, the scenario comparison can support a structured conversation about possible consequences of delayed treatment. For the wider research project, the component creates the bridge between disease analysis and cultivation/harvest decision support.

3.13 Commercialization Findings

The commercialization concept proposed in the original project is feasible in principle because the artifact is modular and API-based. A future product could expose basic yield estimation freely while reserving disease-scenario analytics and historical decision support for premium users. Institutional licensing could provide a route for agricultural cooperatives and extension offices. However, commercialization evidence should

ultimately include user validation, cost modelling, data governance and measurable value to the customer rather than relying only on a proposed pricing model.

3.14 Chapter Summary

The results demonstrate that the individual artifact exists and contains the intended prediction, scenario and presentation layers. The analysis also identifies important validation gaps that must be resolved before the system is described as fully validated. The next chapter summarizes the contribution, limitations and future work and provides a final conclusion aligned with the SLIIT dissertation requirements.

4. CONCLUSION

This research addressed the problem of converting paddy field and disease information into disease-aware harvest decision support. The individual component was developed within Smart Agriculture Decision Support System and focused on disease-aware harvest yield estimation. The implemented artifact combines a Random Forest base-yield model, an XGBoost disease-loss model, deterministic scenario calculations, a Flask API and a Flutter interface.

4.1 Summary of the Research

The research began with the observation that farmers can receive separate information about environmental conditions, disease and markets without receiving a single harvest-oriented interpretation. The literature review established the relevance of smart agriculture, machine learning, IoT and disease severity modelling. The research gap was framed as an integration problem: disease information must be translated into yield impact and presented in an interpretable decision-support workflow.

4.2 Achievement of the Aim

The research aim was to design, implement and evaluate a disease-aware paddy harvest yield estimation and treatment decision-support component. The design and implementation objectives are evidenced by the source repository: the component contains trained model artifacts, feature-construction functions, a prediction endpoint, scenario equations and a Flutter interface. Full empirical evaluation, however, requires the final model metrics, runtime tests and usability evidence to be inserted from the completed experiment records.

4.3 Individual Contribution

The contribution of IT22266446 is the harvest/yield decision layer. It is distinct from the IoT, crop recommendation/market and disease detection components developed by the other group members. The individual artifact receives or is intended to receive field and

disease inputs, estimates baseline yield and disease-related loss, produces scenario values and exposes the result through a reusable interface. This modular contribution provides a clear integration point for the wider DSS.

4.4 Key Technical Contribution

The principal technical contribution is the separation of baseline yield prediction from disease-loss prediction. This allows the system to preserve a baseline estimate while applying a disease-specific loss estimate. The scenario engine then provides an interpretable transformation from model outputs to expected untreated and treated yields. The API boundary makes the analytical service reusable by different clients.

4.5 Limitations and Required Corrections

Before final deployment, the fixed 60% loss-reduction assumption should be replaced or scientifically justified. The treatment decision should incorporate treatment cost, expected price and uncertainty if it is to be presented as financial decision support. The Flutter request payload should be aligned with the backends' required environmental fields. Final model metrics, API latency, integration tests and usability results must be documented. These corrections do not invalidate the artifact; they define the work required to move from a working prototype to a fully validated decision-support product.

4.6 Future Work

Future work should collect larger Sri Lankan field datasets covering additional districts, varieties and seasons. Disease-loss models should be calibrated using observed outbreak and treatment records. Treatment efficacy should be modelled by disease, severity and intervention type rather than by one fixed percentage. The economic layer should calculate expected recovered revenue, treatment cost, net benefit and uncertainty. The system should also support model monitoring, periodic retraining, confidence intervals, multilingual explanations and secure cloud deployment.

4.7 Final Conclusion

The research demonstrates that disease-aware harvest estimation can be implemented as a modular analytical layer within a smart-agriculture DSS. The combination of field variables, disease severity, machine-learning regression and scenario modelling creates a pathway from raw agricultural information to harvest-oriented decision support. The current implementation provides a strong software foundation, while the remaining validation and economic-calibration tasks are necessary for reliable real-world agricultural deployment. The final thesis should therefore present both the achieved implementation and the evidence-based boundaries of what the system can currently claim.

REFERENCES

[1] Department of Census and Statistics, Sri Lanka, “Paddy Statistics—Maha 2023/2024 and Yala 2024,” Ministry of Agriculture, Colombo, Sri Lanka, 2024. [Online]. Available: https://www.statistics.gov.lk/Agriculture/StaticalInformation/PaddyStatistics

[2] International Rice Research Institute, Standard Evaluation System for Rice, 5th ed. Los Baños, Philippines: IRRI, 2013.

[3] N. Mandal et al., “Spectral characterization and severity assessment of rice blast disease using univariate and multivariate models,” Frontiers in Plant Science, vol. 14, 2023, Art. no. 1067189, doi: 10.3389/fpls.2023.1067189.

[4] J. S. A. N. W. Premachandra and P. P. N. V. Kumara, “eKeth: A Machine Learning-Based Mobile Platform to Facilitate the Paddy Cultivation Process in Sri Lanka,” in Proc. 6th Int. Conf. Information Technology Research (ICITR), 2021, pp. 1–6, doi: 10.1109/ICITR54349.2021.9657468.

[5] G. Amarnath et al., “GeoGoviya: Smart Farming Platform for Climate-Smart Agriculture in Sri Lanka,” International Water Management Institute, Colombo, Sri Lanka, 2022.

[6] Department of Agriculture, Sri Lanka, “Krushi Advisor: Agricultural Advisory Mobile Application,” National Agriculture Information and Communication Centre, 2023. [Online]. Available: https://doa.gov.lk/

[7] Y. Wang, U. S. R. Dhamodharan, N. Sarwar, F. A. Almalki, and D. Mohan, “A hybrid approach for rice crop disease detection in agricultural IoT system,” Discover Sustainability, vol. 5, Art. no. 99, 2024, doi: 10.1007/s43621-024-00280-5.

[8] W. Chen et al., “Deep learning-based identification and severity assessment of rice blast disease,” Computers and Electronics in Agriculture, 2024.

[9] A. Gumma et al., “Satellite-based assessment of crop growth stages in paddy-dominated Asian lowlands,” International Journal of Remote Sensing, vol. 42, no. 5, pp. 1795–1816, 2021.

[10] D. G. Peng et al., “Modeling the impact of rice blast on yield under climate change scenarios in Asia,” Global Food Security, vol. 28, 2021, Art. no. 100493.

[11] I. Abdurrahman and A. M. Miyim, “Machine Learning Model for Predicting Rice Crop Yield: A Case Study in Hadejia and Auyo, Nigeria,” UMYU Scientifica, vol. 4, no. 1, pp. 239–249, 2025.

[12] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[13] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[14] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625.

[15] O. Friha, M. A. Ferrag, L. Shu, L. Maglaras, and X. Wang, “Internet of Things for the future of smart agriculture: A comprehensive survey of emerging technologies,” IEEE/CAA Journal of Automatica Sinica, vol. 8, no. 4, pp. 718–752, 2021, doi: 10.1109/JAS.2021.1003925.

[16] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, “Big data in smart farming—A review,” Agricultural Systems, vol. 153, pp. 69–80, 2017, doi: 10.1016/j.agsy.2017.01.023.

[17] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine learning in agriculture: A review,” Sensors, vol. 18, no. 8, Art. no. 2674, 2018, doi: 10.3390/s18082674.

[18] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018, doi: 10.1016/j.compag.2018.02.016.

GLOSSARY

Term	Description
Base yield	Estimated harvest yield before applying the disease-loss adjustment.
Disease-aware prediction	Prediction that explicitly incorporates disease information such as disease type and severity.
Disease-loss percentage	Predicted proportional reduction in yield attributed to the disease condition.
Decision-support system	A software system that combines data, models and rules to assist human decisions.
Design Science Research	A research paradigm focused on constructing and evaluating useful artifacts.
Feature engineering	Transformation of raw variables into the representation expected by a machine-learning model.
Maha	One of Sri Lanka's principal paddy cultivation seasons.
Yala	One of Sri Lanka's principal paddy cultivation seasons.
Random Forest	An ensemble of decision trees used for classification or regression.
Scenario engine	The deterministic software layer that converts model outputs into healthy, untreated and treated scenarios.

Treatment efficacy assumption	The fixed 60% disease-loss reduction used by the current software implementation.
XGBoost	An optimized gradient-boosting framework based on decision trees.

APPENDICES

APPENDIX A – IMPLEMENTED DECISION-SUPPORT FLOW

The following appendix summarizes the verified processing sequence represented in the inspected source code. The base-yield branch constructs a 15-column feature matrix and invokes the Random Forest artifact. The disease-loss branch constructs an eight-column feature matrix and invokes the XGBoost artifact. The two outputs are passed to the deterministic scenario calculations.

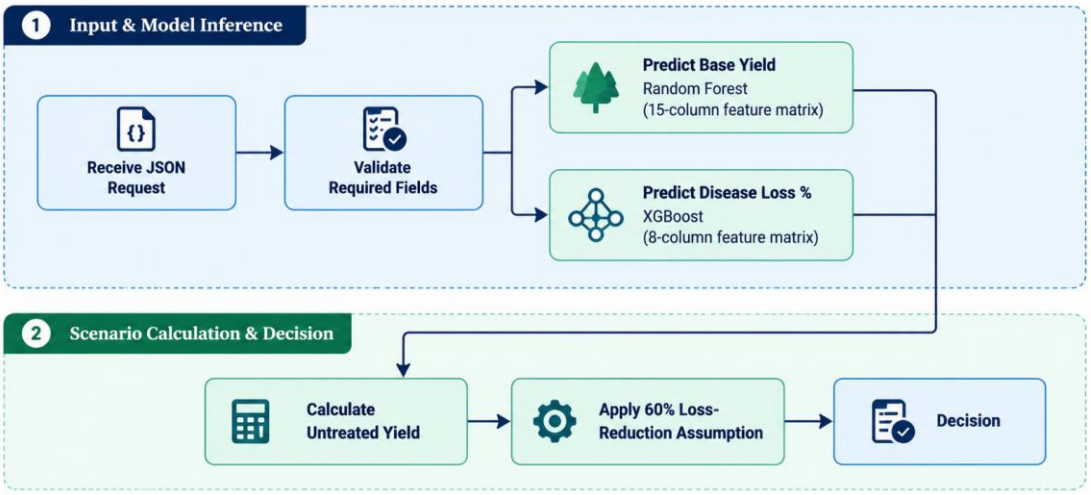


Figure A.1: Decision - support processing flow

APPENDIX B – SOURCE-CODE EVIDENCE: MODEL LOADING AND INPUT CONSTRUCTION

app.py

R26-IT-066-main > backend-python > Decision_support > app.py

```
9   # ========================
10  # LOAD MODELS (FINAL VERSION)
11  # ========================
12
13  model1_bundle = joblib.load(base_dir / "random_forest_model2.pkl")
14  model1 = model1_bundle["model"]
15  cols1 = model1_bundle["columns"]
16
17  model2_bundle = joblib.load(base_dir / "model2_xgboost.pkl")
18  model2 = model2_bundle["model"]
19  cols2 = model2_bundle["columns"]
20
21  print("✅ Models loaded successfully")
22  print("Model1 features:", len(cols1))
23  print("Model2 features:", len(cols2))
24
```

Figure B.1: Model loading logic from the decision - support backend

The source code loads the trained Random Forest and XGBoost model artifacts together with their expected feature-column lists. This allows the backend to construct input DataFrames using the feature order required by each model.

APPENDIX C – SOURCE-CODE EVIDENCE: SCENARIO CALCULATIONS

app.py

R26-IT-066-main > backend-python > Decision_support > app.py

```
 93  def predict():
 94      try:
119          # -------- CALCULATIONS --------
120          untreated_yield = base_yield * (1 - loss_percent / 100)
121
122          # assume treatment reduces loss by 60%
123          treated_loss = loss_percent * 0.4
124          treated_yield = base_yield * (1 - treated_loss / 100)
125
126          decision = "TREAT" if treated_yield > untreated_yield else "DO NOT TREAT"
127
```

Figure C.1: Scenario calculation logic

The source code calculates untreated yield using the predicted disease-loss percentage and calculates treated yield using a fixed loss-reduction assumption. The resulting yield estimates are compared to generate the treatment-related decision output. The 0.4 factor represents an implementation assumption that reduces the predicted loss by 60%; it does not establish experimentally validated treatment effectiveness.